14 | 把经验变成 Skill：让 Claude Code 自动按流程走

Robert · Claude Code 企业级全链路开发实战



你好，我是 Robert。

13 讲我们做完了 Provider 模块，从供应商分析、数据模型设计、后端 8 个任务到前端对接，半天交付。在继续做下一个模块之前，我们先停下来做两件事。

第一件事：回头看看 13 讲里真正决定模块质量的是什么。**不是代码，是代码之前的那些判断——支持哪些供应商、鉴权信息怎么存、健康状态要不要独立成表。这些判断靠的是领域知识。**

第二件事：13 讲的交付流程是固定的——咨询→设计→拆解→执行→前端对接→验收。后面做 Agent、对话引擎、MCP，每个模块都是这套流程。**既然固定，为什么不把它告诉 Claude Code，让它以后自动按流程走**？

这一讲我们就解决这两个问题。

领域理解：被低估的瓶颈

回头看 13 讲，Claude Code 写代码很快，8 个任务两三个小时全部交付。但真正决定这个模块做成什么样的，不是代码，而是代码之前的那些判断——一期支持哪些供应商、要不要引入 LangChain4j、鉴权信息怎么存、健康检查怎么做。这些判断没有一个是 Claude Code 替我做的。它给选项，我做取舍。

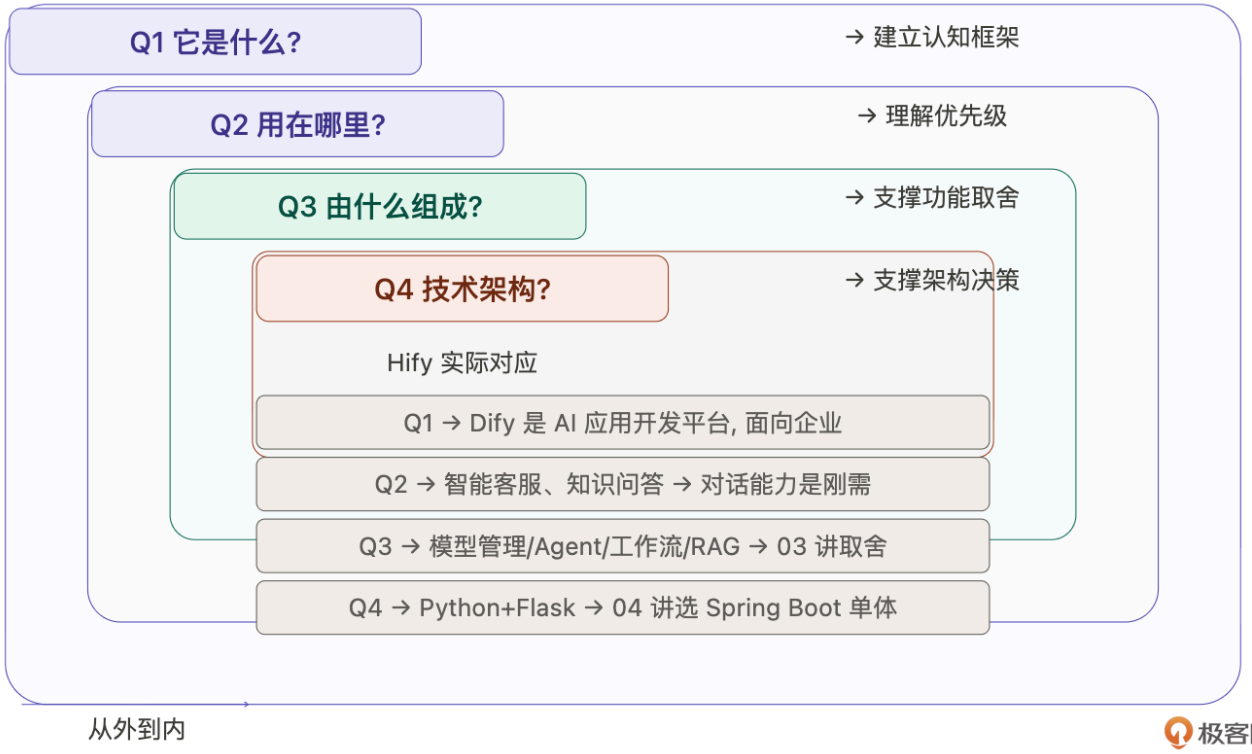
如果这些判断做反了会怎样？引入 LangChain4j，你多了一层重依赖和学习成本。鉴权信息用固定列存，后面加新供应商就得改表。健康状态放在 Provider 表里，高频探测和业务读竞争锁。每个选择单独看都有道理，合在一起就是一个越来越难维护的系统。

取舍的依据是什么？**是你对这个领域的理解。Claude Code 让执行成本趋近于零之后，领域理解的权重不是降低了，而是大幅提升了**。

好消息是，领域知识不是只能靠经验慢慢积累。**Claude Code 本身就是一个极好的领域学习工具**，关键是你要问对问题。

领域快速理解四问

我在动手做 Hify 之前，用 Claude Code 跑了一轮系统性的领域梳理。不是漫无目的地问“Dify 是什么”，而是按一个固定结构问。



第一问：**它是什么，解决什么问题**？建立认知框架。Dify 是 AI 应用开发平台，让不会写代码的人也能搭建 AI 应用。这决定了产品定位，它是平台型产品，不是开发者工具。

第二问：**用在哪里，什么场景**？理解优先级。企业用 Dify 主要做智能客服、内部知识问答、文档处理。这决定了对话能力和工具接入是刚需，工作流编排是进阶需求。13 讲从 Provider 开始做，就是因为理解了场景后知道：没有模型管理，后面所有功能都没有基础。

第三问：**由什么组成，哪些是必要的**？支撑功能取舍。Claude Code 列出模型管理、Agent、工作流、RAG、对话、工具接入等模块。追问“哪些是必须有的”，它帮你区分核心和外围，03 讲的功能取舍直接基于这一步。

第四问：**技术架构是怎样的**？支撑架构决策。了解到 Dify 后端是 Python + Flask，用 Celery 做异步任务——不是照抄，而是理解它为什么这么选，然后根据自己的约束做不同选择。04 讲选 Spring Boot 模块化单体，就是在理解了 Dify 架构复杂度之后的判断。

四个问题从外到内，一两个小时建立 70% 的领域认知，剩下 30% 靠亲手用一下产品、翻一下文档来补。

这套方法不只适用于 Dify。你要做订单系统、监控平台、数据管道，都是同样的路径，先用四问建立全局认知，再进入产品定义和架构设计。拿到任何一个陌生项目，先跑一轮四问，你就知道该做什么、不做什么、先做什么。

Skill：让 Claude Code 教你

回到 13 讲的交付过程。**你有没有发现，整个流程是固定的**？咨询→设计→拆解→执行→前端对接→验收。后面做 Agent、对话引擎、MCP，每个模块都是这五步。既然流程固定，每次都手动给 Claude Code 描述一遍就是浪费。

Claude Code 有一个 Skill 机制，专门解决这个问题。但与其我来讲概念，不如让 Claude Code 自己教你。

第一步：让 Claude Code 教你 Skill 是什么

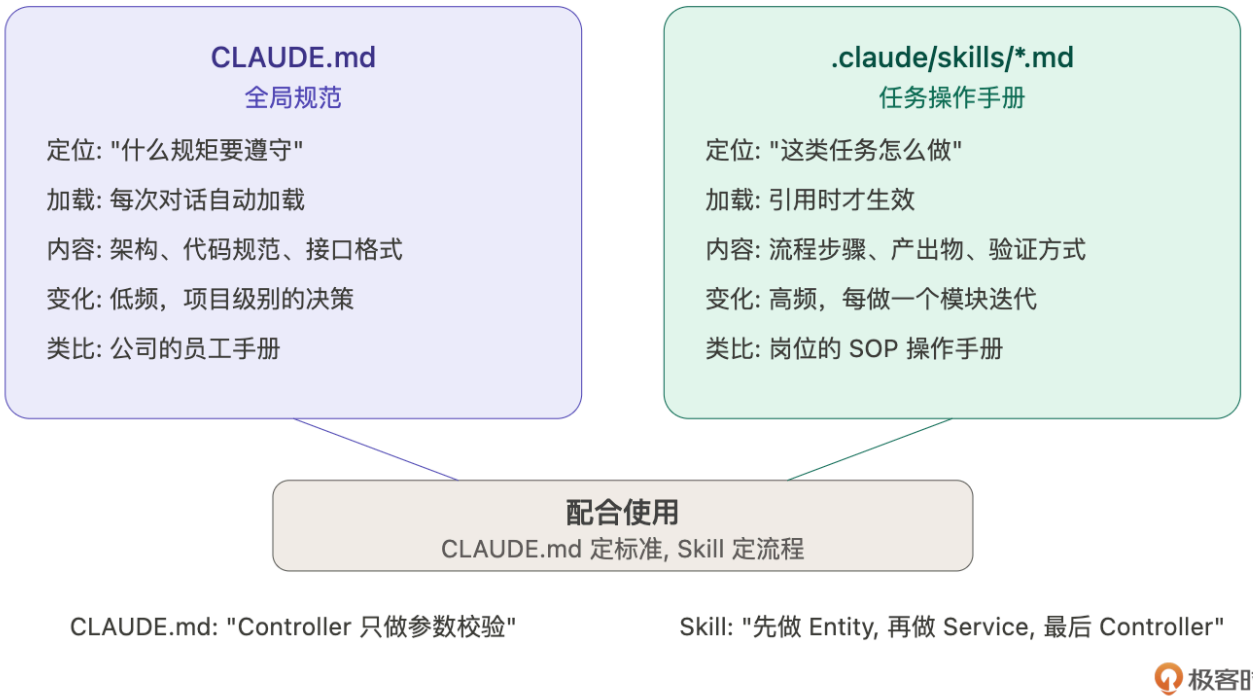
提示词是：

> Claude Code 的 Skill 机制是什么？怎么创建 Skill、怎么使用、Skill 文件放在哪里？和 CLAUDE.md 有什么区别？请详细解释，给我举个例子。

Claude Code 会告诉你：

Skill 是 `.claude/skills/` 目录下的 Markdown 文件，定义特定任务的标准操作流程。你可以理解为给 Claude Code 的“操作手册”。

和 CLAUDE.md 的区别：



CLAUDE.md 是**全局规范，即“什么规矩要遵守”，每次对话自动加载**。Skill 是**具体任务的操作手册，即“这类任务怎么做”，在你引用时才生效**。CLAUDE.md 定义了“Controller 只做参数校验”这种通用规矩，Skill 定义了“做一个新模块应该先梳理需求再拆解再执行”这种具体流程。

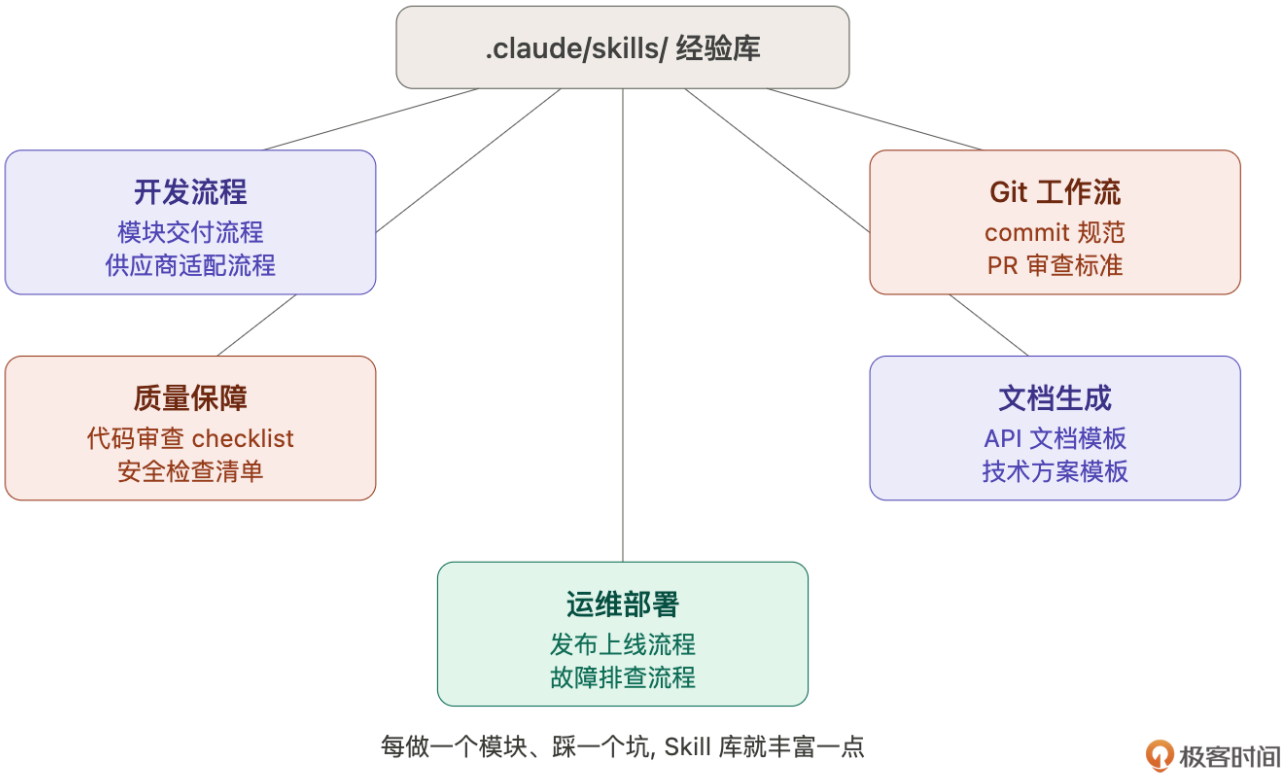
怎么用：给 Claude Code 指令时提到 Skill 的名字，它就会按 Skill 定义的流程执行。比如“按模块交付 Skill 的流程，帮我做 Agent 模块”。

看，你不需要去翻文档学 Skill 的用法，问 Claude Code 它就教你了。这和 10 讲的咨询模式一模一样，不懂就问，它见过的项目比你多。所以，把 AI 当做工具，把 AI 当作老师。

第二步：让 Claude Code 告诉你别人怎么用 Skill 提示词是：

> 业界用 Claude Code Skill 的最佳实践有哪些？大家一般用 Skill 解决什么问题？给我列举一些常见的 Skill 类型和使用场景。

Claude Code 给的回答会打开你的视野，原来 Skill 不只能写开发流程。



开发流程类：新模块交付流程、API 接口开发流程、数据库变更流程。就是我们 13 讲干的事。

质量保障类：代码审查 checklist、单元测试编写规范、安全检查清单。比如每次写 Service 方法都要检查：入参校验了吗？异常处理了吗？缓存失效了吗？日志打了吗？

运维部署类：发布上线流程、环境搭建流程、故障排查流程。把运维经验固化，新人也能按流程操作。

文档生成类：API 文档生成、变更日志生成、技术方案模板。每次写文档不用从零开始，Skill 定义了结构和必填项。

Git 工作流类：commit message 规范、分支管理流程、PR 审查标准。

这些你不需要现在全做，但知道别人怎么用，你对 Skill 的理解会**从“一个功能”变成“一种工作方式”**。Skill 的本质是**把经验编码化——你踩过的坑、总结的流程、做过的判断，全部固化成文字，让 Claude Code 以后自动按你的经验走**。

第三步：让 Claude Code 帮你写 Skill 知道了 Skill 是什么、别人怎么用，现在让 Claude Code 帮你写。

> 我刚完成了 Hify 项目 Provider 模块的开发，流程是这样的：
>
> 1. 先用咨询模式梳理了供应商选型、数据模型设计、边界问题
> 2. 数据模型确定后更新了 schema.sql
> 3. 后端按 MVC 分层拆解：Entity+Mapper → DTO → Service（CRUD+ 连通性测试 + 模型同步 + 健康检查）→ Controller
> 4. 每步编译或 curl 验证通过再进下一步
> 5. 前端对接：创建 API 文件，把 mock 数据源换成真实 API
> 6. 完整验收：后端 curl + 浏览器全流程

> 帮我把这个流程沉淀成一个 Skill 文件，放在 .claude/skills/module-delivery.md。要求：每一步有明确的产出物和验证方式，关键决策点标注“等待用户确认”，把我踩过的坑写成注意事项。

Claude Code 生成第一版，你 review。我的 review 重点：

1. 产出物是否明确？不是“做需求分析”就完了，而是“产出需求分析文档，包含功能范围、数据模型 DDL、设计决策及理由”。Claude Code 需要知道做到什么程度算完。
2. 决策点是否标注？数据模型设计完、后端做完准备做前端之前——这些是你要拍板的地方，Skill 里要写“等待用户确认后再进入下一步”。
3. 踩过的坑有没有写进去？比如，Entity 的 JSON 字段必须用 TypeHandler、schema.sql 要同步更新、前端对接时要更新路由配置。这些是 13 讲实际踩过的，写进 Skill 下次就不会重复踩。

review 完让 Claude Code 改，改完就是你的第一个正式 Skill。生成的 Skill 如下，就不展开说明了，你可以拿着这个 Skill 内容去问 AI 是什么意思：

复制代码

```
# Skill: 业务模块全流程交付

触发方式：当用户说"开发 XX 模块"、"实现 XX 功能"、"交付 XX" 时，按此流程推进。
```

```
---

## 总体原则

- 每步有明确产出物，编译或验证通过再进下一步
- 关键设计决策必须等用户确认，不自行拍板
- 先咨询后实现，先后端后前端

---

## Step 1 — 咨询与设计（不写代码）

**目标**：对齐需求边界和数据模型，避免返工。

**产出物**：
- 候选方案对比表（2-3 个方案，标明取舍）
- 数据模型草稿（表名、核心字段、关联关系）
- 接口清单（Method + Path + 简要说明）

**流程**：
1. 分析业务需求，列出候选技术方案
2. 给出推荐方案及理由
3. 提出需要用户决策的问题（如：是否需要软删除？JSON 字段还是关联表？）

> ⚠️ **等待用户确认**：数据模型和接口设计确认后再进入 Step 2

**注意事项**：
- JSON 字段（如 auth_config）需要用 `@TableName(autoResultMap = true)` + `JacksonTyp
- 高频写入的表（如 health 记录）不要继承 BaseEntity（避免逻辑删除和审计字段的写放大）
- 敏感字段（如 api_key）不能出现在任何响应 DTO 中，用 `authConfigured: boolean` 代替

---

## Step 2 — 更新 schema.sql

**目标**：数据库 DDL 与设计对齐。

**产出物**：
- `hify-app/src/main/resources/db/schema.sql` 新增表 DDL
- `hify-app/src/main/resources/db/schema-h2.sql` H2 兼容版本（JSON → CLOB）

**验证**：
```bash
# 用 mock profile 启动，H2 会自动执行 schema-h2.sql
java -jar hify-app/target/hify-app-0.0.1-SNAPSHOT.jar --spring.profiles.active=mo
# 访问 H2 控制台确认表已创建
open http://localhost:8080/h2-console
```
```

注意事项:
- H2 不支持 `JSON` 类型，必须用 `CLOB` 替代，且两个 schema 文件都要维护
- 索引命名规范：`idx_{表名}_{字段名}`
- 所有外键在应用层维护，不建数据库级外键约束

Step 3 — Entity + Mapper

目标：ORM 层与数据库表对齐。

产出物:
- `entity/XxxEntity.java`（继承 BaseEntity 或独立）
- `mapper/XxxMapper.java`（继承 BaseMapper，复杂查询加 `@Select`）

验证:
```bash
mvn clean install -DskipTests -pl hify-{module} -am
```

注意事项:
- 有 JSON 字段的 Entity 必须加 `@TableName(autoResultMap = true)`，字段上加 `@TableFi
- MyBatis-Plus 3.5.9 分页插件在独立模块 `mybatis-plus-jsqlparser`，缺少会导致 `Paginat
- 自定义查询方法返回 `Optional<T>` 时用 `@Select` + default 方法封装

Step 4 — DTO

目标：定义请求/响应对象，隔离内部实体。

产出物:
- `dto/XxxCreateRequest.java`（`@Valid` 校验注解）
- `dto/XxxUpdateRequest.java`
- `dto/XxxQueryRequest.java`（分页参数继承或包含 page/pageSize）
- `dto/XxxDetailResponse.java`（静态工厂方法 `from(entity, ...)`）

注意事项:
- 响应 DTO 不能暴露 authConfig / password 等敏感字段
- 分页响应统一用 `PageResult.of(list, total, page, pageSize)`，返回 `Result<PageResu
  - ⚠️ 不要让 `PageResult` 继承 `Result`，否则序列化后 `data` 字段是数组，`total` 在外层
- `PageResult` 正确结构：`{ "data": { "list": [...], "total": N, "page": 1, "pageS

Step 5 — Service（业务逻辑）

目标：实现核心业务，接口与实现分离。

产出物:
- `service/XxxService.java`（接口）
- `service/impl/XxxServiceImpl.java`（实现）

流程:
1. CRUD 基础逻辑（含唯一性校验、级联查询）
2. 特殊业务逻辑（如连通性测试、健康检查）
3. 缓存注解（`@Cacheable` / `@CacheEvict`）

验证:
```bash
mvn clean install -DskipTests -pl hify-{module} -am
```

注意事项:
- 跨模块调用走 Service 接口，不直接引用其他模块的 Mapper 或 Entity
- 外部 HTTP 调用（如 LLM API 连通性测试）必须设超时，用 `LlmHttpClient` 的 `get(url, hea
- 健康检查定时任务加 `@ConditionalOnProperty(name = "hify.health-check.enabled", hav
- 使用 `@Qualifier("llmExecutor")` 注入线程池，禁止 `new Thread()` 或默认线程池

Step 6 — Controller

目标：暴露 REST 接口，只做参数校验和 Service 调用。

产出物:
- `controller/XxxController.java`

验证:
```bash
mvn clean install -DskipTests
java -jar hify-app/target/hify-app-0.0.1-SNAPSHOT.jar --spring.profiles.active=mo
```

逐条跑 curl:
```bash
# 创建
curl -s -X POST http://localhost:8080/api/v1/{resource} \
  -H 'Content-Type: application/json' \
  -d '{...}' | jq .

# 列表
curl -s 'http://localhost:8080/api/v1/{resource}?page=1&pageSize=10' | jq .

# 详情
curl -s http://localhost:8080/api/v1/{resource}/1 | jq .
```

```
# 更新
curl -s -X PUT http://localhost:8080/api/v1/{resource}/1 \
  -H 'Content-Type: application/json' \
  -d '{...}' | jq .

# 删除
curl -s -X DELETE http://localhost:8080/api/v1/{resource}/1 | jq .
```

> ⚠️ **等待用户确认**：所有 curl 返回预期结果后再进入前端对接

注意事项：
- Spring Boot 3.2 必须在 `maven-compiler-plugin` 加 `<parameters>true</parameters>`
- Controller 只调用 Service，不写业务逻辑，不直接操作 Mapper

Step 7 — 前端 API 文件

目标：封装后端接口，定义 TypeScript 类型。

产出物：
- `hify-web/src/api/{module}.ts`

内容：
- 请求/响应类型定义（与后端 DTO 字段对齐）
- 导出各接口方法（使用 `request.ts` 的 get/post/put/del）

注意事项：
- 前端 `request.ts` 拦截器会自动解包 `response.data.data`，API 方法的返回类型直接写业务数
- 列表接口返回类型写 `PageResult<T>`（包含 list/total/page/pageSize），对应后端解包后的

Step 8 — 前端页面对接

目标：替换 mock 数据，接入真实 API。

产出物：
- 更新 `views/{module}/XxxList.vue`

流程：
1. 把 HifyTable 的 `api` prop 换成真实 API 方法
2. 表单提交换成 create/update API
3. 删除换成 delete API + useConfirm
4. 按需添加操作按钮（如测试连接）
5. 按需添加状态列（健康状态、关联数量等）

验证：

```bash
# 确保后端已启动
java -jar hify-app/target/hify-app-0.0.1-SNAPSHOT.jar --spring.profiles.active=mo

# 启动前端
cd hify-web && npm run dev
```

在浏览器 DevTools → Network 确认：
- 请求打到了后端（状态码 200，非 ERR_CONNECTION_REFUSED）
- 响应 `data.list` 是数组，`data.total` 是数字
- 表格有数据渲染（或显示"暂无数据"而非一直转圈）

> ⚠️ 如果页面一直转圈：先看 Network 标签确认请求状态码，再排查后端是否启动

注意事项：
- Vite 代理：`/api` → `http://localhost:8080`，前端 baseURL 设为 `/api`，后端路径 `/a
- 前端 `env.d.ts` 不要写 `declare module '*.vue' { ... }`，会覆盖 Volar 的真实类型推断

常见坑速查

现象	原因	修复
页面一直转圈	后端未启动 / 请求 pending	先看 Network 状态码
列表有数据但 total=0 不显示分页	PageResult 继承 Result 导致 data 是数组，total 在外层
@PathVariable 400 错误	缺少 `-parameters` 编译参数	pom.xml compiler plugin 加
JSON 字段反序列化 null	缺少 autoResultMap=true 或 JacksonTypeHandler	Entity 加
mock profile 启动失败 Bean 冲突	RedisConfig 未排除	加 `@Profile("!mock")`
H2 启动报 SQL 错误	schema.sql 用了 MySQL 专属语法（如 JSON 类型）	维护独立 schema-
hify-common 改动后运行旧代码	spring-boot:run 用了旧 jar	改 hify-common 后必须先

然后用同样的方式让它写第二个 Skill，供应商适配：

> 13 讲的连通性测试里，不同供应商的 API 差异用了 if-else，后来我用策略模式重构了。以后加新供应商的流程是固定的：分析 API → 实现 Adapter → 注册到 Factory → 验证。帮我把这个流程也写成 Skill，放在 .claude/skills/provider-adapter.md。

provider-adapter.md 完整内容如下：

复制代码

```
# Skill: 新增 Provider Adapter

触发方式：当用户说"接入新供应商"、"新增 XX 提供商支持"、"加一个 Adapter" 时按此流程推进。

---

## 背景

Provider 的连通性测试、模型同步、调用逻辑按供应商类型有差异。
最初用 switch-case 实现，后来重构为策略模式：

```
ProviderAdapterFactory
  └── Map<ProviderType, ProviderAdapter>
        ├── OpenAiAdapter        (OPENAI / OPENAI_COMPATIBLE / DEEPSEEK)
        ├── AnthropicAdapter     (ANTHROPIC)
        ├── AzureOpenAiAdapter   (AZURE_OPENAI)
        └── OllamaAdapter        (OLLAMA)
```

每个 Adapter 实现统一接口，Factory 按类型路由，新增供应商只需加一个 Adapter 类 + 注册，不改

Step 1 — 分析目标供应商 API

目标：搞清楚接入该供应商需要哪些差异化实现。

需要调研的问题（逐一回答）：

问题	说明
认证方式	Bearer Token / API Key Header / 双 Header / 无认证？
列模型接口	URL 路径？返回结构（`data[]` / `models[]` / 其他）？
必填 authConfig 字段	如 `apiKey`、`apiVersion`、`anthropicVersion`
baseUrl 默认值	官方默认是什么？用户可否自定义？
特殊请求头	如 Anthropic 的 `anthropic-version`
Chat 调用路径	`/v1/chat/completions` 还是其他？
流式响应格式	SSE `data: {...}` 标准格式，还是自定义格式？

产出物：一份简短的 API 特征说明（口头或注释均可）

> ⚠️ **等待用户确认**：API 特征分析结果确认后再写代码

Step 2 — 实现 Adapter

目标：新建一个实现 `ProviderAdapter` 接口的类。
```

接口定义（位于 `hify-provider/.../adapter/ProviderAdapter.java`）：

```java
public interface ProviderAdapter {
    /** 该Adapter支持的供应商类型（可多个） */
    List<String> supportedTypes();

    /** 连通性测试，返回延迟和模型数 */
    ConnectionTestResult test(Provider provider, OkHttpClient testClient);

    /** 拉取并返回模型列表（用于同步） */
    List<String> listModels(Provider provider, OkHttpClient client);

    /** 构造chat请求体（流式） */
    RequestBody buildChatRequest(Provider provider, List<ChatMessage> messages);

    /** 解析流式响应的一行delta文本，无内容返回null */
    String parseDelta(String line);
}
```

文件位置：`hify-provider/src/main/java/com/hify/provider/adapter/impl/XxxAdapte

实现模板：

```java
@Component
public class XxxAdapter implements ProviderAdapter {

    private final ObjectMapper objectMapper;

    @Override
    public List<String> supportedTypes() {
        return List.of("XXX");
    }

    @Override
    public ConnectionTestResult test(Provider provider, OkHttpClient testClient)
        long start = System.currentTimeMillis();
        try {
            String apiKey = getAuth(provider, "apiKey");
            String url = provider.getBaseUrl().stripTrailing() + "/v1/models";
            Map<String, String> headers = Map.of("Authorization", "Bearer " + api

            String body = llmHttpClient.get(url, headers, testClient);
            int latency = (int) (System.currentTimeMillis() - start);
            int modelCount = parseDataArraySize(body);
            return ConnectionTestResult.ok(latency, modelCount);
```

```
        } catch (LlmApiException e) {
            return ConnectionTestResult.fail(e.getMessage());
        } catch (Exception e) {
            return ConnectionTestResult.fail("测试异常: " + e.getMessage());
        }
    }

    // ... 其他方法
}
```

注意事项:
- `getAuth(provider, key)` 找不到字段时抛 `IllegalArgumentException("authConfig 缺少
- 解析模型列表时不同供应商返回字段不同: OpenAI 是 `data[].id`, Ollama 是 `models[].name`
- 流式解析: OpenAI 格式每行是 `data: {...}`, 遇到 `data: [DONE]` 停止; Anthropic 是 `da
- 有特殊 Header 的 (如 Anthropic `anthropic-version`) 放在 authConfig 里, 不要硬编码版本

验证:
```bash
mvn clean install -DskipTests -pl hify-provider -am
```

Step 3 — 注册到 Factory

目标: 让 Factory 能路由到新 Adapter。

Factory 实现 (位于 `hify-provider/.../adapter/ProviderAdapterFactory.java`):

```java
@Component
public class ProviderAdapterFactory {

    private final Map<String, ProviderAdapter> adapterMap;

    // Spring自动注入所有ProviderAdapter实现
    public ProviderAdapterFactory(List<ProviderAdapter> adapters) {
        this.adapterMap = new HashMap<>();
        for (ProviderAdapter adapter : adapters) {
            for (String type : adapter.supportedTypes()) {
                adapterMap.put(type.toUpperCase(), adapter);
            }
        }
    }

    public ProviderAdapter get(String type) {
        ProviderAdapter adapter = adapterMap.get(type.toUpperCase());
        if (adapter == null) {
```

```
            throw new BizException(ErrorCode.PROVIDER_TYPE_NOT_SUPPORTED);
        }
        return adapter;
    }
}
```

注册方式：新 Adapter 加 `@Component` 注解，`supportedTypes()` 返回对应的类型字符串，

验证：启动后在日志里确认 adapterMap 包含新类型（可在 Factory 构造方法加一行 log）。

Step 4 — 更新 ProviderType 枚举（如需要）

如果新供应商需要在前端下拉菜单里出现，同步更新：

- 后端：`hify-provider/.../constant/ProviderType.java`（如果有枚举）
- 前端：`hify-web/src/views/provider/ProviderList.vue` 的 `providerTypes` 数组
- 数据库：`provider.type` 是 varchar，无需迁移，直接用新字符串值

Step 5 — 验证

后端 curl 验证
```bash
# 1. 创建新供应商
curl -s -X POST http://localhost:8080/api/v1/providers \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "测试-XXX",
    "type": "XXX",
    "baseUrl": "https://api.xxx.com",
    "authConfig": { "apiKey": "sk-test-xxx" }
  }' | jq .

# 2. 连通性测试（id替换为上一步返回的id）
curl -s -X POST http://localhost:8080/api/v1/providers/1/test-connection | jq .
```

预期：
- 真实 key：`success: true`，有 `latencyMs` 和 `modelCount`
- 假 key：`success: false`，`errorMessage` 包含"无效"或"认证失败"，**不能是 500**

浏览器验证
1. 前端下拉能选到新类型
2. 创建后列表显示正常
3. 点"测试"按钮有结果提示

```
197
198 ---
199
200 ## 常见坑
201
202 | 现象 | 原因 | 修复 |
203 |------|------|------|
204 | Factory 找不到新 Adapter | 忘加 `@Component` 或 `supportedTypes()` 返回值大小写不一致
205 | authConfig 字段缺失导致 500 | 前端创建时没传必填的 auth 字段 | `getAuth()` 的异常信息要
206 | 连通性测试超时 | 用了默认 OkHttpClient（无超时限制） | 必须用注入的 `testClient`（10s 超
207 | 模型数量永远是 0 | 响应结构解析错误（字段名不是 `data`） | 用 `objectMapper.readTree(bo
208 | 流式响应乱码/截断 | Anthropic 等有自己的 SSE 事件类型，直接用 OpenAI 解析逻辑会漏掉 | `p
209
```

在我们写了两个 Skill 后，你就会发现：

1. **原来 Skill 就是写 MarkDown 文档。**
2. **Skill 就是按格式定规范，在 MarkDown 里面写好规范，让 Claude Code 去识别执行。**

接下来我们实际跑一遍 Skill，看一下它的用途。

实际跑一遍：用 Skill 启动 Agent 模块 Skill 写好了，当场验证。给 Claude Code：

> 按模块交付 Skill 的流程，帮我做 Agent 管理模块。先从第一步开始，梳理 Agent 模块的需求和数据模型。

Claude Code 读到 Skill 后，自动按流程走。它会问你 Agent 模块的核心功能是什么、涉及哪些关联（绑定模型、绑定工具）、数据模型怎么设计，然后给出需求分析文档，标注“等待用户确认”。

对比没有 Skill 时：你要手写一大段指令描述整个流程，先帮我分析需求，然后设计数据模型，然后按 Entity、DTO、Service、Controller 的顺序拆解……现在一句话就够了。

而且 Skill 保证了流程一致性，不管是你自己做还是团队里其他人做，引用同一个 Skill，产出的代码结构和质量标准是一样的。

我不会在这一讲展开 Agent 模块的具体实现（那是下一讲的内容），但你已经看到了 Skill 驱动开发的效果：你的经验在 Skill 里积累，Claude Code 按你的经验走，你只需要在关键决策点拍板。

Skill 也需要迭代

最后提一点：第一版 Skill 不会完美。

你用模块交付 Skill 做了 Agent 模块，可能发现 Skill 里没提“跨模块依赖怎么处理”——Agent 依赖 Provider 和 MCP，这在 13 讲做 Provider 时没遇到。把这个补进 Skill。

做了对话引擎，可能发现 Skill 里的后端拆解不适用于流式响应场景，需要加一条“如果涉及 SSE 流式响应，Service 层用 SseEmitter + llmExecutor”。补进去。

Skill 和 CLAUDE.md 一样是活文档。02 讲说的 SDD 闭环——定规范→AI 执行→发现问题→迭代规范——在 Skill 上同样适用。每做一个模块，Skill 就更完善一点。做到第四五个模块的时候，你的 Skill 已经覆盖了绝大多数场景，Claude Code 几乎不需要额外指导就能按你的标准交付。

用 Skill 思维重构 if-else

13 讲连通性测试里，不同供应商的 API 差异用了 if-else 处理。当时说“先跑通，下一讲重构”。现在来做。

重构本身不复杂，用策略模式替代 if-else：

> 重构 Provider 模块的连通性测试。当前是 if-else 按 type 分发，改成策略模式：
>
> 1. 定义 ProviderAdapter 接口：testConnection(provider)、listModels(provider)
> 2. 实现四个适配器：OpenAiAdapter、AnthropicAdapter、OllamaAdapter、OpenAiCompatibleAdapter（和 OpenAiAdapter 共用逻辑）
> 3. 创建 ProviderAdapterFactory：根据 provider.type 返回对应的 Adapter 实例

> 4. ProviderService 里的 if-else 替换为 factory.getAdapter(provider.getType()).testConnection(provider)
> 5. OpenAiCompatibleAdapter 直接继承 OpenAiAdapter，不需要额外代码

这个指令我就不分析输出了，因为重要的是这个问题和指令本身。

重构完之后，思考一个问题：以后加新供应商（比如 Gemini），流程是什么？ 写一个 GeminiAdapter 实现 ProviderAdapter 接口，在 Factory 里注册——就这两步。不需要改任何已有代码。

这个“加新供应商”的流程也可以沉淀成 Skill：



如上图所示，我们不用去写一行代码，这个 Skill 也沉淀下来了。所以你看，Skill 不只是大流程，小流程也可以沉淀。大的“模块交付 Skill”覆盖通用的模块开发流程，小的“供应商适配 Skill”覆盖特定场景的操作步骤。积累下来，你的 `.claude/skills/` 目录就是一个越来越丰富的经验库。

总结

这一讲做了三件事：**讲清楚领域理解的重要性**、**让 Claude Code 教你 Skill 并帮你写 Skill**、**用策略模式重构 if-else 并沉淀适配 Skill**。

也就是从方法论上总结，**我们应该怎么思考问题，应该让我们的思考到执行模板化**。这里展开说一下，AI 给我们带来的最大的效率提升，是它能帮我们做很多模板化的事情。有一个经验是：**脏活累活，我们嫌烦没有技术含量的，都是适合 AI 做的**。

两个方法论输出：

领域快速理解四问。进入一个陌生领域时用——是什么、用在哪里、由什么组成、技术架构怎样。一两个小时建立 70% 的认知，支撑产品定义和架构决策。

Skill 沉淀。不需要你自己研究 Skill 怎么写——让 Claude Code 教你概念、告诉你别人怎么用、帮你把经验写成 Skill 文件。你只需要做两件事：把你的实际流程描述给它，以及 review 它生成的 Skill 是否准确。Skill 也是活文档，每做一个模块就迭代一次，越来越完善。

从下一讲开始，每个模块都用 Skill 驱动开发。你会明显感觉到节奏变快了，不是因为 Claude Code 变聪明了，而是你的经验在 Skill 里持续积累，它按你的经验走，你只在关键决策点拍板。

思考题

回顾你过去做项目的经验，有没有哪些“每次都要做、每次都差不多”的流程？比如接入一个新的第三方 SDK、搭建一个新的微服务、做一轮性能测试。选一个，试着把它写成一个 Skill。Skill 应该覆盖：第一步做什么、第二步做什么、每步的输出是什么、怎么验证。写完之后让 Claude Code 按这个 Skill 执行一次，看看效果如何。

期待你的分享！如果今天的课程让你有所收获，也欢迎转发给有需要的朋友，邀请他来一起学习，我们下节课再见！

© 版权归极客邦科技所有，未经许可不得传播售卖。 页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (12)

最新　精选



神经蛙

2026-04-19 来自江西

老师 实际的 单元测试，页面测试 是让AI 自己跑吗？ 需要提供哪些变量给他，目前这步感觉AI落地很麻烦； 自己要手动做很多步骤， 包括接口调用，权限认证； 表单点击 等等一系列

作者回复: 这块在最后有专门的章节讲解的。 这是27 讲《27 | AI 写代码，保证质量不靠运气，靠体系》的开头部分，你可以先看看思路：

Anthropic 公开说过：Claude Code 自身的代码，全部由 AI 编写，工程师只负责 review。在我看来，这句话有一个没说出来的前提：他们敢这么做，不是因为 AI 不会出错，而是因为有一套完整的质量保障体系。

在说这套体系之前，我想先说清楚一个判断。

现在很多新闻告诉我们：写好 SDD，直接扔给 AI，它跑一个晚上就把整个项目搞定了。我的个人体感是：做不到，至少现在做不到。

不是 AI 能力不够，而是这句话偷换了一个概念：保证代码质量的不是 AI，是体系。AI 只是体系的执行者。

这两件事的区别很大。

● "AI 帮你保证质量"的意思是：你把代码交给 AI，它会自动发现所有问题，自动写出正确的测试，自动保证系统稳定。这是不现实的。AI 写代码时专注于功能实现，它不会主动质疑自己的安全模型，不会想到"这个接口的权限校验是不是有漏洞"，不会考虑并发场景下的数据一致性。

● "体系保证质量，AI 协助执行"的意思是：你先建立质量保障体系，识别核心链路、划定测试边界、制定 review 策略，然后让 AI 在这个体系里高效执行。体系是你建的，AI 是执行者。

一个项目的质量由四个层次组成。

1. 核心链路识别：哪些是系统的核心路径，哪些是风险集中的地方。这个判断必须由人来做，AI 辅助分析。做错了，后面所有测试都是在错误的地方使劲。

2. 单元测试：覆盖纯逻辑，不依赖外部系统。这部分独立性强、模式固定，AI 能规划得很好，可以大比例交给 AI 执行。但要注意：AI 偏逻辑完备，测试用例容易过多，你要把握边界，砍掉不必要的。

3. 集成测试：功能维度的端到端验证，从 Controller 到数据库的完整链路。mock 策略要你来定，执行让 AI 来做，一个场景跑通再写下一个。

4. 混沌测试：故障注入验证韧性，是质量体系的天花板。单测和集成测试验证"正常情况下对不对"，混沌测试验证"出问题时垮不垮"，节点宕机、网络分区、磁盘打满、慢消费者积压。这一层在项目维护阶段才有意义，但要从架构阶段就知道它在哪里。

所以，正确的工作方式不是"告诉 AI：给我写测试用例，保证代码不出问题"然后等它跑完。而是你来拆解项目，划定边界，AI 在你划定的边界里执行。

这一讲围绕这个思路展开。文章分两部分：方法论说清楚怎么想，实操给你能直接用的提示词和流程。




4

风之翼
2026-06-23 来自四川

在实践上有个问题想请教一下老师：看老师对skill的用法是将自己独立写这个模块的思路沉淀成skill，来指导AI一步步编写。在实践中我把模块的设计方案（包括表结构、er图、接口详情、业务需求）整理成一个skill，每次改动这个模块的时候自动加载skill中的内容，这样是不是思路偏了。另外一个问题是：每次不用加载所有的代码，只加载skill文档即可了解，会不会减少token的使用。请老师指导一下。

> 作者回复:“在实践中我把模块的设计方案（包括表结构、er图、接口详情、业务需求）整理成一个skill”。
> 我觉得你这个思路没偏，在我看来这才是最合理的方式。就是把课程的内容知识内化完，最终形成自己的skill。这样你在后续的使用中就不用从头来一遍。
>
> “自己形成skill”隐藏的好处就是：你已经内化了这套思路，说明了这个skill不是死的，随着你的理解、经验的加深，你有持续更新这个skill的能力。这点是最关键的。
>
> 这门课本身目的就是形成一个skill。但是分为这么多步去理解项目，核心是为了让你理解这个skill怎么来的。知其然知其所以然。
>
> “另外一个问题是：每次不用加载所有的代码，只加载skill文档即可了解，会不会减少token的使用。请老师指导一下。”
> 这点是肯定的，doc/skill/claude.md 某种意义上来说就是context、memory的一种。经过提炼的内容本身就是会降低token的消耗。

1

Geek_40827e
2026-05-24 来自北京

老师您好，感觉您这门课不仅仅能学到AI编程的知识和方法，也连带着把java后端工程项目的一些实践经验学到了，感谢感谢。
这里我请教个问题啊，类似阿里巴巴java开发手册，effective java这样的内容，是更合适做为claude.md还是skill呢，期待您解惑，感谢。

> 作者回复: 感谢认可哈～
> 在我看来，”类似阿里巴巴java开发手册，effective java这样的内容“这些更适合作为skill、cc的hook等一些限制。
> 你可以去学习一下harness，“阿里巴巴java开发手册，effective java”这些属于限制和规范类。在不同的工具下限制的方法和工具不一样。

1



MoeRookie
2026-05-12 来自山东

还请老师把skills提交到github下?

作者回复: 收到～ 项目代码会尽快更新🌹

1



Oliveira
2026-04-18 来自北京

老师有代码吗?

作者回复: https://github.com/socutes/hify

1



水中鱼儿
2026-06-15 来自浙江

老师，源码中没有看到skill文件，是还没有更新进去吗

作者回复: 代码没更新，我马上更新～，skill是最后加的，忘了提交～



pcc
2026-05-08 来自湖北

我生成的第一个skill感觉只有步骤，少了很多限制和规则，这部分是老师自己根据经验补的吗?

作者回复: 我没有手动补充过skill的内容。我是自动生成的是crud的skill。这个skill 我已经放在代码仓库了。你用的是哪个模型，生成的skill方便发一下嘛? 一起看下

Geek_33d5cb
2026-05-01 来自湖南

文中这里出现错误
“如上图所示，我们不用去写一行代码，这个 Skil 也沉淀下来了。”
应该是 Skill
看来是没用使用 AI 做审查~

作者回复: 说明不是用AI写的～～～。我来改一下






Geek_57ddfe
2026-04-20 来自北京

老师,您本地的 .claude/skills下文件夹的内容能提交到GitHub下吗?

作者回复: 可以的






神经蛙
2026-04-19 来自江西

老师的代码仓库文件中 不会附带有 这些基础的 提示词， skill ，让我们自己通过 与 CC的交互方式 也来快速实现吗?

作者回复: 提示词的话，都在课程里面了，可以直接拿。 项目中skill，claude.md 这些该有的都有的。你可以去看一下源码

共 2 条评论>

